

Database Schema Reference - JNX-OS v2

Overview

Database: PostgreSQL 15 (Supabase)
Schema Version: v2.0
Migration Status: ✔ Complete

Quick Stats

- **Tables:** 8 core tables + 1 billing table
 - **Indexes:** 23 optimized indexes
 - **Foreign Keys:** 6 relationships
 - **Auto-Update Triggers:** 3 `updated_at` triggers
-

Table of Contents

- 1. [Core Tables](#)
 - 2. [Billing Tables](#)
 - 3. [Qryx Tables](#)
 - 4. [Relationships](#)
 - 5. [Indexes](#)
 - 6. [Triggers](#)
 - 7. [Queries](#)
 - 8. [Migrations](#)
-

Core Tables

1. `orgs`

Purpose: Stores organization/company information (multi-tenant architecture)

Columns:

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key
clerk_org_id	TEXT	YES	NULL	Clerk organization ID (nullable for default orgs)
name	TEXT	NO	-	Organization name
slug	TEXT	YES	NULL	URL-friendly identifier
created_at	TIMESTAMPTZ	NO	NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp

Constraints:

```
PRIMARY KEY (id)
UNIQUE (clerk_org_id) -- Only if not NULL
```

Indexes:

```
CREATE INDEX idx_orgs_clerk_org_id ON orgs(clerk_org_id);
CREATE INDEX idx_orgs_slug ON orgs(slug);
```

Auto-Update Trigger:

```
CREATE TRIGGER update_orgs_updated_at
BEFORE UPDATE ON orgs
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();
```

Sample Data:

```
INSERT INTO orgs (id, name, slug) VALUES
('550e8400-e29b-41d4-a716-446655440000', 'JNX Labs', 'jnxlabs'),
('550e8400-e29b-41d4-a716-446655440001', 'Acme Corp', 'acme-corp');
```

2. users

Purpose: Stores user accounts with Clerk integration

Columns:

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key
clerk_user_id	TEXT	NO	-	Clerk user ID (unique)
org_id	UUID	NO	-	Foreign key to orgs
email	TEXT	NO	-	User email address
first_name	TEXT	YES	NULL	User's first name
last_name	TEXT	YES	NULL	User's last name
role	TEXT	NO	'member'	User role (admin, member)
deleted_at	TIMESTAMPTZ	YES	NULL	Soft delete timestamp (GDPR)
created_at	TIMESTAMPTZ	NO	NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp

Constraints:

```

PRIMARY KEY (id)
UNIQUE (clerk_user_id)
FOREIGN KEY (org_id) REFERENCES orgs(id) ON DELETE CASCADE

```

Indexes:

```

CREATE INDEX idx_users_clerk_user_id ON users(clerk_user_id);
CREATE INDEX idx_users_org_id ON users(org_id);
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_deleted_at ON users(deleted_at);
CREATE INDEX idx_users_role ON users(role);

```

Auto-Update Trigger:

```
CREATE TRIGGER update_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();
```

Sample Data:

```
INSERT INTO users (clerk_user_id, org_id, email, first_name, last_name, role) VALUES
('user_abc123', '550e8400-e29b-41d4-a716-446655440000', 'john@jnxlabs.ai', 'John', 'Doe', 'admin'),
('user_xyz789', '550e8400-e29b-41d4-a716-446655440000', 'jane@jnxlabs.ai', 'Jane', 'Smith', 'member');
```

3. audit_logs

Purpose: Tracks all user actions for security and compliance (GDPR requirement)

Columns:

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key
user_id	UUID	YES	NULL	Foreign key to users (NULL for system events)
org_id	UUID	YES	NULL	Foreign key to orgs
action	TEXT	NO	-	Action performed (e.g., 'user.login', 'subscription.created')
resource_type	TEXT	YES	NULL	Type of resource (e.g., 'user', 'subscription')
resource_id	TEXT	YES	NULL	ID of affected resource
metadata	JSONB	YES	NULL	Additional context
ip_address	TEXT	YES	NULL	User's IP address
user_agent	TEXT	YES	NULL	Browser/client info
created_at	TIMESTAMPTZ	NO	NOW()	Event timestamp

Constraints:

```

PRIMARY KEY (id)
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
FOREIGN KEY (org_id) REFERENCES orgs(id) ON DELETE CASCADE

```

Indexes:

```

CREATE INDEX idx_audit_logs_user_id ON audit_logs(user_id);
CREATE INDEX idx_audit_logs_org_id ON audit_logs(org_id);
CREATE INDEX idx_audit_logs_action ON audit_logs(action);
CREATE INDEX idx_audit_logs_created_at ON audit_logs(created_at DESC);
CREATE INDEX idx_audit_logs_resource ON audit_logs(resource_type, resource_id);

```

Sample Data:

```

INSERT INTO audit_logs (user_id, org_id, action, resource_type, resource_id,
ip_address) VALUES
(
  (SELECT id FROM users WHERE email = 'john@jnxlabs.ai'),
  (SELECT org_id FROM users WHERE email = 'john@jnxlabs.ai'),
  'user.login',
  'user',
  'user_abc123',
  '192.168.1.1'
);

```

4. system_events

Purpose: Logs system-level events (errors, health checks, integrations)

Columns:

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid() ()	Primary key
event_type	TEXT	NO	-	Event type (e.g., 'web-hook.stripe.success')
severity	TEXT	NO	'info'	Severity (debug, info, warn, error, critical)
message	TEXT	NO	-	Human-readable message
metadata	JSONB	YES	NULL	Additional event data
created_at	TIMESTAMPTZ	NO	NOW() ()	Event timestamp

Constraints:

```
PRIMARY KEY (id)
CHECK (severity IN ('debug', 'info', 'warn', 'error', 'critical'))
```

Indexes:

```
CREATE INDEX idx_system_events_type ON system_events(event_type);
CREATE INDEX idx_system_events_severity ON system_events(severity);
CREATE INDEX idx_system_events_created_at ON system_events(created_at DESC);
```

Sample Data:

```
INSERT INTO system_events (event_type, severity, message, metadata) VALUES
('webhook.stripe.received', 'info', 'Stripe webhook processed successfully', '{"event_id": "evt_abc123"}'),
('database.migration', 'info', 'Migrated to schema v2.0', '{"version": "2.0"}');
```

Billing Tables

5. `billing_subscriptions`

Purpose: Stores Stripe subscription data for Qryx SaaS billing

Columns:

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key
clerk_user_id	TEXT	NO	-	Clerk user ID (owner of subscription)
shop_domain	TEXT	NO	-	Shopify shop domain
stripe_customer_id	TEXT	NO	-	Stripe customer ID
stripe_subscription_id	TEXT	NO	-	Stripe subscription ID (unique)
plan_id	TEXT	NO	-	Plan identifier (starter, professional, business)
status	TEXT	NO	-	Subscription status (active, canceled, past_due)
current_period_start	TIMESTAMPTZ	YES	NULL	Billing period start
current_period_end	TIMESTAMPTZ	YES	NULL	Billing period end
cancel_at_period_end	BOOLEAN	NO	FALSE	Will cancel at end of period
canceled_at	TIMESTAMPTZ	YES	NULL	Cancellation timestamp
created_at	TIMESTAMPTZ	NO	NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp

Constraints:


```

PRIMARY KEY (id)
UNIQUE (stripe_subscription_id)
CHECK (plan_id IN ('starter', 'professional', 'business'))
CHECK (status IN ('active', 'canceled', 'past_due', 'trialing', 'incomplete'))

```

Indexes:

```

CREATE INDEX idx_billing_clerk_user ON billing_subscriptions(clerk_user_id);
CREATE INDEX idx_billing_shop ON billing_subscriptions(shop_domain);
CREATE INDEX idx_billing_stripe_customer ON billing_subscriptions(stripe_customer_id);
CREATE INDEX idx_billing_stripe_subscription ON billing_subscriptions(stripe_subscription_id);
CREATE INDEX idx_billing_status ON billing_subscriptions(status);

```

Auto-Update Trigger:

```

CREATE TRIGGER update_billing_subscriptions_updated_at
BEFORE UPDATE ON billing_subscriptions
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

```

Trigger Function:

```

CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

Sample Data:

```

INSERT INTO billing_subscriptions (
    clerk_user_id, shop_domain, stripe_customer_id, stripe_subscription_id,
    plan_id, status, current_period_start, current_period_end
) VALUES (
    'user_abc123',
    'shopbotv3.myshopify.com',
    'cus_abc123',
    'sub_abc123',
    'starter',
    'active',
    NOW(),
    NOW() + INTERVAL '30 days'
);

```

Plan Details:

Plan ID	Price	Conversations	Stripe Price ID
starter	\$29/month	500	price_1SjkKKBQ5QFS35pBxGKE0r50
professional	\$79/month	2,000	price_1SjkQTBQ5QFS35pBpWkdi5ws
business	\$199/month	5,000	price_1Sjkr4BQ5QFS35pBkhTJsxk2

Qryx Tables

6. qryx_shops

Purpose: Stores Shopify shop OAuth tokens and configuration

Columns:

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key
shop_domain	TEXT	NO	-	Shopify shop domain (unique)
access_token	TEXT	NO	-	Encrypted OAuth access token
scope	TEXT	NO	-	OAuth scopes granted
status	TEXT	NO	'active'	Shop status (active, inactive, suspended)
installed_at	TIMESTAMPTZ	NO	NOW()	Installation timestamp
uninstalled_at	TIMESTAMPTZ	YES	NULL	Uninstallation timestamp
created_at	TIMESTAMPTZ	NO	NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp

Constraints:

```
PRIMARY KEY (id)
UNIQUE (shop_domain)
CHECK (status IN ('active', 'inactive', 'suspended'))
```

Indexes:

```
CREATE INDEX idx_qryx_shops_domain ON qryx_shops(shop_domain);
CREATE INDEX idx_qryx_shops_status ON qryx_shops(status);
```

Sample Data:

```
INSERT INTO qryx_shops (shop_domain, access_token, scope, status) VALUES
('shopbotv3.myshopify.com', 'shpat_abc123...encrypted',
'read_products,read_customers', 'active');
```

7. qryx_conversations

Purpose: Tracks chat conversations for usage billing

Columns:

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key
shop_domain	TEXT	NO	-	Shopify shop domain
conversation_id	TEXT	NO	-	Unique conversation identifier
customer_id	TEXT	YES	NULL	Shopify customer ID (if known)
message_count	INTEGER	NO	0	Number of messages in conversation
started_at	TIMESTAMPTZ	NO	NOW()	Conversation start time
ended_at	TIMESTAMPTZ	YES	NULL	Conversation end time
created_at	TIMESTAMPTZ	NO	NOW()	Creation timestamp

Constraints:

```
PRIMARY KEY (id)
UNIQUE (conversation_id)
```

Indexes:

```
CREATE INDEX idx_qryx_conversations_shop ON qryx_conversations(shop_domain);
CREATE INDEX idx_qryx_conversations_started_at ON qryx_conversations(started_at DESC);
```

Sample Data:

```
INSERT INTO qryx_conversations (shop_domain, conversation_id, message_count) VALUES
('shopbotv3.myshopify.com', 'conv_abc123', 5);
```

8. qryx_config

Purpose: Stores chatbot configuration per shop

Columns:

Column	Type	Nullable	Default	Description
id	UUID	NO	gen_random_uuid()	Primary key
shop_domain	TEXT	NO	-	Shopify shop domain (unique)
enabled	BOOLEAN	NO	TRUE	Chatbot enabled/disabled
theme	JSONB	NO	{}	Theme configuration (colors, position, etc.)
behavior	JSONB	NO	{}	Behavior settings (auto-open, sound, etc.)
created_at	TIMESTAMPTZ	NO	NOW()	Creation timestamp
updated_at	TIMESTAMPTZ	NO	NOW()	Last update timestamp

Constraints:

```
PRIMARY KEY (id)
UNIQUE (shop_domain)
```

Indexes:

```
CREATE INDEX idx_qryx_config_shop ON qryx_config(shop_domain);
```

Sample Data:

```
INSERT INTO qryx_config (shop_domain, theme, behavior) VALUES
('shopbotv3.myshopify.com',
 '{"primaryColor": "#3b82f6", "position": "bottom-right"}',
 '{"autoOpen": false, "soundEnabled": true}')
);
```

Relationships

Entity Relationship Diagram

```

orgs
├── users (1:N)
│   ├── audit_logs (1:N)
│   └── billing_subscriptions (1:N via clerk_user_id)
└── audit_logs (1:N)

billing_subscriptions
├── gryx_shops (1:1 via shop_domain)
│   ├── gryx_conversations (1:N)
│   └── gryx_config (1:1)

```

Foreign Key Constraints

```

-- users → orgs
ALTER TABLE users ADD CONSTRAINT fk_users_org
    FOREIGN KEY (org_id) REFERENCES orgs(id) ON DELETE CASCADE;

-- audit_logs → users
ALTER TABLE audit_logs ADD CONSTRAINT fk_audit_logs_user
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL;

-- audit_logs → orgs
ALTER TABLE audit_logs ADD CONSTRAINT fk_audit_logs_org
    FOREIGN KEY (org_id) REFERENCES orgs(id) ON DELETE CASCADE;

-- billing_subscriptions → users (via clerk_user_id, not enforced at DB level)
-- Enforced at application level due to TEXT type mismatch

```

Indexes

Performance Indexes

Total: 23 indexes across all tables

Critical Indexes:

```
-- Authentication lookups
CREATE INDEX idx_users_clerk_user_id ON users(clerk_user_id);
CREATE INDEX idx_users_email ON users(email);

-- Billing queries
CREATE INDEX idx_billing_clerk_user ON billing_subscriptions(clerk_user_id);
CREATE INDEX idx_billing_shop ON billing_subscriptions(shop_domain);
CREATE INDEX idx_billing_status ON billing_subscriptions(status);

-- Shop lookups
CREATE INDEX idx_qryx_shops_domain ON qryx_shops(shop_domain);

-- Audit trail queries
CREATE INDEX idx_audit_logs_created_at ON audit_logs(created_at DESC);
CREATE INDEX idx_audit_logs_user_id ON audit_logs(user_id);

-- GDPR soft deletes
CREATE INDEX idx_users_deleted_at ON users(deleted_at);
```

Index Verification

```
-- List all indexes
SELECT
    schemaname,
    tablename,
    indexname,
    indexdef
FROM pg_indexes
WHERE schemaname = 'public'
ORDER BY tablename, indexname;

-- Check index usage
SELECT
    schemaname,
    tablename,
    indexname,
    idx_scan as index_scans,
    idx_tup_read as tuples_read,
    idx_tup_fetch as tuples_fetched
FROM pg_stat_user_indexes
WHERE schemaname = 'public'
ORDER BY idx_scan DESC;
```

Triggers

Auto-Update Triggers

Purpose: Automatically update `updated_at` timestamp on record modification.

Trigger Function:

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Applied To:

```
-- orgs table
CREATE TRIGGER update_orgs_updated_at
BEFORE UPDATE ON orgs
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

-- users table
CREATE TRIGGER update_users_updated_at
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();

-- billing_subscriptions table
CREATE TRIGGER update_billing_subscriptions_updated_at
BEFORE UPDATE ON billing_subscriptions
FOR EACH ROW
EXECUTE FUNCTION update_updated_at_column();
```

Common Queries

User Management

Get user by Clerk ID:

```
SELECT u.*, o.name as org_name
FROM users u
JOIN orgs o ON u.org_id = o.id
WHERE u.clerk_user_id = 'user_abc123'
AND u.deleted_at IS NULL;
```

Get all users in organization:

```
SELECT u.email, u.first_name, u.last_name, u.role
FROM users u
WHERE u.org_id = (SELECT id FROM orgs WHERE slug = 'jnxlabs')
AND u.deleted_at IS NULL
ORDER BY u.created_at DESC;
```

Soft delete user (GDPR):


```
UPDATE users
SET deleted_at = NOW()
WHERE clerk_user_id = 'user_abc123';
```

Billing Queries

Get active subscriptions:

```
SELECT
  bs.shop_domain,
  bs.plan_id,
  bs.status,
  bs.current_period_end,
  u.email as user_email
FROM billing_subscriptions bs
JOIN users u ON u.clerk_user_id = bs.clerk_user_id
WHERE bs.status = 'active'
ORDER BY bs.created_at DESC;
```

Get subscription by shop:

```
SELECT *
FROM billing_subscriptions
WHERE shop_domain = 'shopbotv3.myshopify.com'
AND status = 'active'
LIMIT 1;
```

Revenue analytics:

```
SELECT
  plan_id,
  COUNT(*) as subscription_count,
  CASE
    WHEN plan_id = 'starter' THEN COUNT(*) * 29
    WHEN plan_id = 'professional' THEN COUNT(*) * 79
    WHEN plan_id = 'business' THEN COUNT(*) * 199
  END as monthly_revenue
FROM billing_subscriptions
WHERE status = 'active'
GROUP BY plan_id;
```

Expiring subscriptions (next 7 days):

```
SELECT
  bs.shop_domain,
  bs.plan_id,
  bs.current_period_end,
  u.email
FROM billing_subscriptions bs
JOIN users u ON u.clerk_user_id = bs.clerk_user_id
WHERE bs.status = 'active'
AND bs.current_period_end BETWEEN NOW() AND NOW() + INTERVAL '7 days'
ORDER BY bs.current_period_end;
```

Qryx Shop Queries

Get shop with subscription:

```
SELECT
  qs.shop_domain,
  qs.status as shop_status,
  bs.plan_id,
  bs.status as subscription_status,
  qc.enabled as chatbot_enabled
FROM qryx_shops qs
LEFT JOIN billing_subscriptions bs ON bs.shop_domain = qs.shop_domain AND bs.status =
'active'
LEFT JOIN qryx_config qc ON qc.shop_domain = qs.shop_domain
WHERE qs.shop_domain = 'shopbotv3.myshopify.com';
```

Get conversation stats:

```
SELECT
  shop_domain,
  COUNT(*) as total_conversations,
  SUM(message_count) as total_messages,
  AVG(message_count) as avg_messages_per_conversation
FROM qryx_conversations
WHERE started_at >= NOW() - INTERVAL '30 days'
GROUP BY shop_domain;
```

Usage tracking (current billing period):

```
SELECT
  bs.shop_domain,
  bs.plan_id,
  CASE
    WHEN bs.plan_id = 'starter' THEN 500
    WHEN bs.plan_id = 'professional' THEN 2000
    WHEN bs.plan_id = 'business' THEN 5000
  END as conversation_limit,
  COUNT(qc.id) as conversation_count,
  CASE
    WHEN bs.plan_id = 'starter' THEN 500 - COUNT(qc.id)
    WHEN bs.plan_id = 'professional' THEN 2000 - COUNT(qc.id)
    WHEN bs.plan_id = 'business' THEN 5000 - COUNT(qc.id)
  END as conversations_remaining
FROM billing_subscriptions bs
LEFT JOIN qryx_conversations qc
  ON qc.shop_domain = bs.shop_domain
  AND qc.started_at >= bs.current_period_start
  AND qc.started_at < bs.current_period_end
WHERE bs.shop_domain = 'shopbotv3.myshopify.com'
  AND bs.status = 'active'
GROUP BY bs.shop_domain, bs.plan_id, bs.current_period_start, bs.current_period_end;
```

Audit & Monitoring

Recent user activity:

```
SELECT
  al.action,
  al.resource_type,
  al.created_at,
  u.email
FROM audit_logs al
JOIN users u ON u.id = al.user_id
WHERE al.user_id = (SELECT id FROM users WHERE email = 'john@jnxlabs.ai')
ORDER BY al.created_at DESC
LIMIT 50;
```

System errors (last 24 hours):

```
SELECT *
FROM system_events
WHERE severity IN ('error', 'critical')
      AND created_at >= NOW() - INTERVAL '24 hours'
ORDER BY created_at DESC;
```

Webhook events:

```
SELECT
  event_type,
  COUNT(*) as count,
  MAX(created_at) as last_occurred
FROM system_events
WHERE event_type LIKE 'webhook.%'
      AND created_at >= NOW() - INTERVAL '7 days'
GROUP BY event_type
ORDER BY count DESC;
```

Migrations

Migration History

Version	Date	Description	File
v1.0	Dec 25, 2025	Initial schema	schema.sql
v2.0	Dec 28, 2025	Added Clerk integra- tion, GDPR columns	schema-v2.sql
v2.1	Dec 29, 2025	Added billing_subscriptions table	MIGRA- TION_SIMPLE.sql

Running Migrations

Via Supabase Dashboard:

1. Navigate to **SQL Editor**
2. Paste migration SQL
3. Click **Run**
4. Verify with `SELECT * FROM users LIMIT 1;`

Via Command Line:

```
# Using psql
psql "postgres://postgres:[PASSWORD]@[HOST]:5432/postgres" -f MIGRATION_SIMPLE.sql

# Using Supabase CLI
supabase db push
```

Schema Verification

Check table exists:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public'
AND table_name = 'billing_subscriptions';
```

Check columns:

```
SELECT
  column_name,
  data_type,
  is_nullable,
  column_default
FROM information_schema.columns
WHERE table_name = 'billing_subscriptions'
ORDER BY ordinal_position;
```

Check constraints:

```
SELECT
  conname as constraint_name,
  contype as constraint_type,
  pg_get_constraintdef(oid) as definition
FROM pg_constraint
WHERE conrelid = 'billing_subscriptions'::regclass;
```

Backup & Recovery

Backup Strategies

Supabase (Automatic):

- Daily automated backups (retained for 7 days)

- Point-in-time recovery available
- Accessed via Supabase Dashboard → Database → Backups

Manual Backup:

```
# Full database dump
pg_dump "postgresql://postgres:[PASSWORD]@[HOST]:5432/postgres" > backup_$(date +%Y%m%d).sql

# Specific tables
pg_dump -t billing_subscriptions -t qryx_shops "postgresql://..." > billing_backup.sql

# Schema only
pg_dump --schema-only "postgresql://..." > schema_backup.sql
```

Restore

```
# Full restore
psql "postgresql://..." < backup_20251229.sql

# Specific table
psql "postgresql://..." -c "DROP TABLE IF EXISTS billing_subscriptions CASCADE;"
psql "postgresql://..." < billing_backup.sql
```

Performance Tuning

Slow Query Analysis

```
-- Enable query logging
ALTER DATABASE postgres SET log_min_duration_statement = 1000; -- Log queries > 1s

-- Find slow queries
SELECT
  calls,
  total_time,
  mean_time,
  query
FROM pg_stat_statements
WHERE mean_time > 100 -- More than 100ms average
ORDER BY total_time DESC
LIMIT 10;
```

Index Optimization

```
-- Find missing indexes
SELECT
  schemaname,
  tablename,
  attname,
  n_distinct,
  correlation
FROM pg_stats
WHERE schemaname = 'public'
  AND n_distinct > 100 -- High cardinality
  AND correlation < 0.1 -- Low correlation
ORDER BY n_distinct DESC;

-- Create composite index if needed
CREATE INDEX idx_composite_shop_status
ON billing_subscriptions(shop_domain, status);
```

Table Statistics

```
-- Update statistics for query planner
ANALYZE billing_subscriptions;

-- Check table bloat
SELECT
  schemaname,
  tablename,
  pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) AS size
FROM pg_tables
WHERE schemaname = 'public'
ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;
```

Security

Row-Level Security (RLS)

Supabase supports RLS. Example policies:

```
-- Enable RLS on billing_subscriptions
ALTER TABLE billing_subscriptions ENABLE ROW LEVEL SECURITY;

-- Policy: Users can only see their own subscriptions
CREATE POLICY "Users can view own subscriptions"
ON billing_subscriptions
FOR SELECT
USING (clerk_user_id = auth.uid());

-- Policy: Service role can do anything
CREATE POLICY "Service role has full access"
ON billing_subscriptions
FOR ALL
USING (auth.role() = 'service_role');
```

Encryption

- **At Rest:** Supabase encrypts all data at rest (AES-256)
 - **In Transit:** All connections use SSL/TLS
 - **Access Tokens:** Stored encrypted using application-level encryption
-

Summary

This database schema reference covers:

- ✓ **9 Tables:** Core, billing, and Qryx-specific
- ✓ **23 Indexes:** Optimized for common queries
- ✓ **6 Foreign Keys:** Maintain referential integrity
- ✓ **3 Triggers:** Auto-update timestamps
- ✓ **Common Queries:** Ready-to-use SQL for all use cases
- ✓ **Migration Guide:** Safe schema updates
- ✓ **Performance Tuning:** Index optimization and monitoring
- ✓ **Security:** RLS policies and encryption

Related Documentation:

- [Stripe Setup Guide](#) (/STRIPE_SETUP_GUIDE.md)
 - [API Endpoints Reference](#) (/API_ENDPOINTS_REFERENCE.md)
 - [Testing Guide](#) (/TESTING_GUIDE_PHASE5A.md)
 - [Troubleshooting Guide](#) (/TROUBLESHOOTING_GUIDE.md)
-

Document Version: 1.0

Last Updated: December 29, 2025

Maintained By: JNXLabs Database Team